Conformance testing for implementations of this Recommendation will be conducted within the framework of the Cryptographic Algorithm Validation Program (CAVP) and the Cryptographic Module Validation Program (CMVP). The requirements of this Recommendation are indicated by the word "**shall**". Some of these requirements may be out-of-scope for CAVP and CMVP validation testing, and thus are the responsibility of entities using, implementing, installing, or configuring applications that incorporate this Recommendation.

3 Glossary of Terms, Acronyms and Mathematical Symbols

3.1 Terms and Definitions

Adversary	An entity that is not authorized to access or modify information, or who works to defeat any protections afforded the information.
Algorithm	A clearly specified mathematical process for computation; a set of rules that, if followed, will give a prescribed result.
Approved	FIPS-approved and/or NIST-recommended. An algorithm or technique that is either 1) specified in a FIPS or NIST Recommendation, 2) adopted in a FIPS or NIST Recommendation or 3) specified in a list of NIST-approved security functions.
Approved hash algorithms	Hash algorithms specified in FIPS 180-4.
Bit string	An ordered sequence of 0 and 1 bits. In this Recommendation, the leftmost bit is the most significant bit of the string. The rightmost bit is the least significant bit of the string.
Bits of security	See security strength.
Block cipher	An invertible symmetric-key cryptographic algorithm that operates on fixed-length blocks of input using a secret key and an unvarying transformation algorithm. The resulting output block is the same length as the input block.
Collision	An event in which two different messages have the same message digest.
Collision resistance	An expected property of a hash function whereby it is computationally infeasible to find a collision, See "Collision".

Message digest	The result of applying a hash function to a message. Also known as a “hash value” or “hash output”.
Preimage	A message *X* that produces a given message digest when it is processed by a hash function.
Preimage resistance	An expected property of a hash function such that, given a randomly chosen message digest, *message_digest*, it is computationally infeasible to find a preimage of the *message_digest*, See “Preimage”.
Random bit	A bit for which an attacker has exactly a 50% probability of success of guessing the value of the bit as either a zero or one. It is also called an unbiased bit.
Random bit generator	A device or algorithm that can produce a sequence of bits that appear to be both statistically independent and unbiased.
Randomized hashing	A process by which the input to a hash function is randomized before being processed by the hash function.
Random number	A value in a set of numbers that has an equal probability of being selected from the total population of possibilities and, in that sense, is unpredictable. A random number is an instance of an unbiased random variable, that is, the output produced by a uniformly distributed random process. Random numbers may, e.g., be obtained by converting suitable stings of random bits (see [SP 800-90A], Appendix B.5 for details).
Second preimage	A message *X’*, that is different from a given message *X* , such that its message digest is the same as the known message digest of *X*.
Second preimage resistance	An expected property of a hash function whereby it is computationally infeasible to find a second preimage of a known message digest, See “Second preimage”.
Secret keying material	The binary data that is used to form secret keys, such as AES encryption or HMAC keys.
Security strength (Also “bits of security”)	A number associated with the amount of work (that is, the number of operations) that is required to break a cryptographic algorithm or system. If 2^N execution operations of the algorithm (or system) are required to break the cryptographic algorithm, then the security strength is N bits.
Shall	Used to indicate a requirement of this Recommendation.
Shared secret	A secret value that has been computed using a key agreement algorithm and is used as input to a key derivation function.

the message input to the hash function is the maximum length allowed by the hash function, while the high end of the range applies to the situation where the message input length is relatively small. Information on determining the actual second preimage resistance strengths of the approved hash functions for different message lengths is provided in Appendix A. In the case of SHA-384, SHA-512/224 or SHA-512/256, the second preimage resistance strength does not depend on the message length; details can be found in Appendix A.

Note that the preimage resistance and the second preimage resistance strengths are greater than the collision resistance strength for each of the approved hash algorithms specified in FIPS 180-4.

5 Hash function Usage

5.1 Truncated Message Digest

Some applications may require a value that is shorter than the (full-length) message digest provided by an approved hash function as specified in FIPS 180-4. In such cases, it may be appropriate to use a subset of the bits produced by the hash function as the (shortened) message digest.

Let the (shortened) message digest be called a truncated message digest, and let λ be its desired length in bits. A truncated message digest may be used if the following requirements are met:

1. The length of the output block of the approved hash function to be used **shall** be greater than λ (i.e., $L > \lambda$).
2. The λ left-most bits of the full-length message digest **shall** be selected as the truncated message digest.

   For example, if a truncated message digest of 96 bits is desired, the SHA-256 hash function could be used (e.g., because it is available to the application, and provides an output larger than 96 bits). The leftmost 96 bits of the 256-bit message digest generated by SHA-256 are selected as the truncated message digest, and the rightmost 160 bits of the message digest are discarded.
3. If collision resistance is required, λ **shall** be at least twice the required collision resistance strength s (in bits) for the truncated message digest (i.e., $\lambda \geq 2s$).

These specifications for truncating the output of a cryptographic hash function promote application interoperability in situations where the use of shortened message digests is appropriate (and permissible), as determined by implementers and application developers acting in conformance with NIST Standards and Recommendations.

Truncating the message digest can impact the security of an application. By truncating a message digest, the expected collision resistance strength is reduced from $L/2$ to $\lambda/2$ (in bits). For the example in item 2 above, even though SHA-256 provides 128 bits of collision resistance, the collision resistance provided by the 96-bit truncated message digest is half the length of the truncated message digest, which is 48 bits, in this case.

generation process – even if it is practical to find collisions for the unrandomized hash function.

2) Randomized hashing does not offer the message preparer or the signature verifiers any additional protection from a misbehaving signer (who may, for example, generate one signature that can be associated with either of two chosen messages after finding randomized versions of those messages that hash to the same value).

3) Randomized hashing does not improve the assurance of origin authenticity, data integrity, and/or signatory non-repudiation provided by the digital signature to signature verifiers (i.e., relying parties).

5.3 Keyed-Hash Message Authentication Codes (HMAC)

5.3.1 Description

Message authentication codes (MACs) provide data authentication and integrity protection. Two types of algorithms for computing a MAC have been approved: 1) MAC algorithms that are based on approved block cipher algorithms (more information can be found in [SPs 800-38B, C and D]) and 2) MAC algorithms that are based on hash functions, called HMAC algorithms, that are specified in FIPS 198-1. This section discusses the use of HMAC.

An output from an HMAC algorithm is called an HMAC output. The HMAC output is either used in its entirety, or is truncated (see Section 5.3.3) when it is transmitted for subsequent verification. The transmitted value is called a *MacTag*. The HMAC algorithm requires the use of a secret key that is shared between the entity that generates the HMAC output (e.g., a message sender), and the entity (or entities) that need to verify the transmitted *MacTag* (message receiver(s)).

The HMAC output is generated from a secret key and the string of "text" to be MACed (e.g., a message to be sent) using the HMAC algorithm. The *MacTag* is provided to the *MacTag* verifier, along with the "text" that was MACed (e.g., the sender transmits both the *MacTag* and the message to the intended receiver).

The verifier computes an HMAC output on the received "text" using the same key and HMAC algorithm that were (purportedly) used to generate the received *MacTag*, generates a (new) *MacTag* (either a full or truncated HMAC output), and then compares the verifier-generated *MacTag* with the received *MacTag*. If the two values match, the "text" has been correctly received and the verifier is assured that the entity that generated the *MacTag* is a member of the community of users that share the key.

The security strength provided by the HMAC algorithm depends on the security strength of the HMAC key, the underlying hash algorithm and the length of the *MacTag*.

5.3.2 The HMAC Key

The security strength of the HMAC algorithm depends, in part, on the security strength of the HMAC key, *K*. An HMAC key **shall** have a security strength that meets or exceeds the security strength required to protect the data over which the HMAC is computed.

5.3.5 Security of the HMAC Values

The successful verification of a *MacTag* does not completely guarantee that the accompanying text is authentic; there is a slight chance that an adversary with no knowledge of the HMAC key, *K*, can present a (*MacTag*, *text*) pair that will pass the verification procedure. From the perspective of an adversary that does not know the HMAC key *K* (i.e., the adversary is not among the community of users that share the key), the assurance of authenticity provided by a *MacTag* depends on its length and on the number of failed *MacTag* verifications allowed by a system for each value of the HMAC key.

Let 2^t be the number of failed *MacTag* verifications allowed by a system for each value of the HMAC key. The *MacTag* length, λ, and the value of *t* need to be chosen to avoid an unacceptable probability of falsely accepting forged data. The likelihood of accepting forged data as authentic is $(1/2)^{(\lambda - t)}$. For example, if λ is 32, and a system allows 2^{12} failed *MacTag* verifications for any given value of the HMAC key, then the likelihood of accepting forged data is $(1/2)^{20}$. In order to increase assurance of authenticity, either λ would need to be increased, or the number of allowed failed *MacTag* verifications for each value of the HMAC key would need to be decreased. To avoid having an unacceptable probability of falsely accepting forged data at any time, the value of the HMAC key must be changed to a new value before the number of failed *MacTag* authentications reaches the maximum allowed number (2^t). For the example above, with $\lambda = 32$ and $t = 20$, the HMAC key must be changed before the number of failed *MacTag* verifications reaches 2^{12}.

The table below provides the likelihoods of accepting forged data for different *MacTag* lengths and allowed numbers of MAC verifications using a given value of the HMAC key. This table is intended to assist the implementers of HMAC applications in security-sensitive systems to assess the security risk associated with using *MacTags*.

λ	Number of Failed Verifications Allowed (2^t): 2^{20}	2^{30}	2^{35}
40	2^{-20}	2^{-10}	2^{-5}
64	2^{-44}	2^{-34}	2^{-29}
96	2^{-76}	2^{-66}	2^{-61}

Table 2: Risks/Likelihoods of Accepting Forged Data

Each 2^{-x} entry displayed in Table 2 is the probability of accepting forged data given the indicated choices for the *MacTag* length and the number of failed *MacTag* verifications allowed for a fixed value of the HMAC key. If the probability is not acceptable for the system, the HMAC key **shall** be changed to a new value before the number of failed MAC verifications reaches 2^t.

A commonly acceptable length for the *MacTag* is 64 bits; *MacTags* with lengths shorter than 64 bits are discouraged.